

NOTES

Linear Algebra



Linear Algebra

Introduction to Linear Algebra

- Linear algebra is a branch of mathematics that helps us understand and work with linear equations, transformations, and vector spaces.
- It provides a foundation for solving many practical problems by using a structured, mathematical approach.

Think of it as a set of tools to handle relationships between numbers, vectors, and spaces, enabling us to perform operations like adding, scaling, and rotating data. It is a core concept in fields like machine learning, data science, and computer graphics.

What Problems Does Linear Algebra Solve?

Systems of Equations

- Linear algebra helps solve multiple equations at once, especially when they are interrelated.
- For example, finding how different factors like speed, distance, and time relate to one another.

Data Transformations

- It allows us to rotate, scale, or translate data, making it possible to manipulate and analyze datasets efficiently.

Dimensionality Reduction

- Techniques like **PCA (Principal Component Analysis)** rely on linear algebra to simplify complex datasets by reducing the number of variables without losing essential information.

Why Study Linear Algebra?

Data Science

- Linear algebra forms the backbone of algorithms such as:
 - **Linear Regression:** Used to predict outcomes based on input features.

- **Principal Component Analysis (PCA):** Used for reducing dimensions in large datasets to make them easier to visualize or process.
- It helps in working with large datasets effectively.

Machine Learning

- Many machine learning models are based on matrix operations, which are a key part of linear algebra.
- Techniques like **gradient descent** for optimizing models use linear algebra concepts extensively.

Graphics

- In computer graphics, linear algebra is used to create realistic animations, rotate 3D objects, and manipulate images.
- For instance, scaling an image or rotating a 3D model is handled using linear transformations like **matrix multiplication**.

Vectors and Vector Operations

Definition of a Vector

A **vector** is a quantity that has two key properties: **magnitude** (size or length) and **direction**. Vectors are used to represent quantities that cannot be fully described by a single number, such as force, velocity, or displacement.

Example: The Vector [3, 4] in 2D Space

- The number **3** represents the horizontal movement (along the x-axis).
- The number **4** represents the vertical movement (along the y-axis).

This vector essentially describes a point or position in 2D space starting from the origin ([0, 0]). Its **magnitude** can be calculated using the Pythagorean theorem:

$$\text{Magnitude} = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5$$

So, ([3, 4]) represents a movement with a magnitude of **5 units** at a specific angle from the horizontal axis.

Vector Addition and Subtraction

Addition

Combine two vectors by adding their corresponding elements.

Example: Adding $([3, 4])$ and $([1, 2])$ results in:

$$[3 + 1, 4 + 2] = [4, 6]$$

This operation is like combining two movements into one final movement.

Subtraction

Subtract corresponding elements of one vector from another.

Example: Subtracting $([1, 2])$ from $([3, 4])$ results in:

$$[3 - 1, 4 - 2] = [2, 2]$$

This can be thought of as finding the vector difference or the movement required to go from one position to another.

Python Example

```
In [ ]: import numpy as np

# Define two vectors
vector_a = np.array([3, 4])
vector_b = np.array([1, 2])

# Perform addition and subtraction
vector_add = vector_a + vector_b
vector_sub = vector_a - vector_b

print("Vector Addition:", vector_add)
print("Vector Subtraction:", vector_sub)
```

Vector Addition: $[4\ 6]$

Vector Subtraction: $[2\ 2]$

Explanation

- `np.array([3, 4])` : Creates a vector with elements `1` and `2`.
- `vector_a + vector_b` : Adds corresponding elements of the two vectors.

- `vector_a - vector_b` : Subtracts corresponding elements of the two vectors.

Scalar Multiplication

Scalar multiplication involves multiplying a vector by a scalar to change its magnitude while keeping its direction the same.

What is a Scalar?

A **scalar** is a regular number that represents a quantity with no direction—like `2`, `-3`, or `0.5`. Scalars are used to scale (resize) vectors, either making them larger, smaller, or reversing their direction.

How Does Scalar Multiplication Work?

When you multiply a vector by a scalar, you multiply each element of the vector by that number. This operation changes the **magnitude** (size) of the vector but not its direction (unless the scalar is negative, in which case the direction reverses).

Example: Multiply the Vector `[3, 4]` by a Scalar `2`

$$[3, 4] \times 2 = [3 \times 2, 4 \times 2] = [6, 8]$$

Here, the magnitude of the vector doubles, but it still points in the same direction.

Negative Scalar: Multiply `[3, 4]` by `-1`

$$[3, 4] \times -1 = [3 \times -1, 4 \times -1] = [-3, -4]$$

This reverses the direction of the vector, pointing it in the opposite direction.

Python Example

```
In [ ]: vector_a = np.array([3, 4])
        scalar = 3
        vector_scaled = scalar * vector_a

        print("Scaled Vector:", vector_scaled)
```

Scaled Vector: [9 12]

Explanation

- `scalar * vector_a` : Multiplies each element of `vector_a` by the scalar value.

Dot Product

The **dot product** is a way of combining two vectors to calculate a single number. This number tells us how similar the two vectors are in terms of their direction and size.

Dot Product in Mathematics

In math, the dot product is calculated by multiplying the corresponding elements of two vectors and then adding those products together.

- For two vectors $A = [a_1, a_2, a_3]$ and $B = [b_1, b_2, b_3]$, the formula is:

$$A \cdot B = (a_1 \times b_1) + (a_2 \times b_2) + (a_3 \times b_3)$$

- **Example:**

Let $A = [2, 3]$ and $B = [4, 5]$. Then:

$$A \cdot B = (2 \times 4) + (3 \times 5) = 8 + 15 = 23$$

Geometric Meaning

The dot product can also be used to measure how closely two vectors align:

- If the dot product is **positive**, the vectors point in roughly the same direction.
- If it is **zero**, the vectors are at a right angle (perpendicular).
- If it is **negative**, the vectors point in opposite directions.

Dot Product in Data Science

Similarity Measurement:

The dot product tells us how similar two vectors are. For example:

- If you have two user profiles represented as vectors (like their movie preferences), a higher dot product means the users are more similar.

Linear Regression:

- The dot product is used to calculate predictions by combining input features (vector of data points) with the model weights.

Cosine Similarity:

- In recommendation systems or text analysis, **cosine similarity** (based on the dot product) compares two vectors to measure their similarity regardless of their size.

Python Example

```
In [ ]: vector_a = np.array([3, 4])
vector_b = np.array([1, 2])
dot_product = np.dot(vector_a, vector_b)

print("Dot Product:", dot_product)
```

Dot Product: 11

Explanation:

- `np.dot(vector_a, vector_b)` : Computes the dot product.

Matrices and Matrix Operations

Definition of a Matrix

- A **matrix** is a rectangular arrangement of numbers (or other mathematical objects) organized into **rows** and **columns**.
- The numbers in the matrix are called **elements**.

- Example of a Matrix:**

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

- This matrix has 3 rows and 3 columns, making it a 3x3 matrix.

Matrix Addition and Subtraction

- Matrix addition and subtraction involve adding or subtracting corresponding elements from two matrices of the same size (i.e., having the same number of rows and columns).

- Matrix Addition:** To add two matrices, simply add their corresponding elements.

$$\text{If } A = \begin{pmatrix} a_1 & a_2 \\ a_3 & a_4 \end{pmatrix} \text{ and } B = \begin{pmatrix} b_1 & b_2 \\ b_3 & b_4 \end{pmatrix},$$

then the sum $A + B$ is:

$$A + B = \begin{pmatrix} a_1 + b_1 & a_2 + b_2 \\ a_3 + b_3 & a_4 + b_4 \end{pmatrix}$$

- Example:**

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

$$A + B = \begin{pmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

- Matrix Subtraction:** To subtract one matrix from another, subtract the corresponding elements.

$$A - B = \begin{pmatrix} a_1 - b_1 & a_2 - b_2 \\ a_3 - b_3 & a_4 - b_4 \end{pmatrix}$$

- Example:**

$$A = \begin{pmatrix} 10 & 20 \\ 30 & 40 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 15 \\ 25 & 35 \end{pmatrix}$$

$$A - B = \begin{pmatrix} 10-5 & 20-15 \\ 30-25 & 40-35 \end{pmatrix} = \begin{pmatrix} 5 & 5 \\ 5 & 5 \end{pmatrix}$$

Python Example

```
In [ ]: matrix_a = np.array([[1, 2], [3, 4]])
        matrix_b = np.array([[5, 6], [7, 8]])

# Addition and Subtraction
matrix_add = matrix_a + matrix_b
matrix_sub = matrix_a - matrix_b

print("Matrix Addition:\n", matrix_add)
print("Matrix Subtraction:\n", matrix_sub)
```

Matrix Addition:

```
[[ 6  8]
 [10 12]]
```

Matrix Subtraction:

```
[[ -4 -4]
 [ -4 -4]]
```

Explanation:

- Define two 2x2 matrices using np.array.
- Perform element-wise addition and subtraction.

Matrix Multiplication

Matrix multiplication is a bit more involved than addition or subtraction. It is the operation where you multiply the **rows** of the first matrix by the **columns** of the second matrix, and sum the products of corresponding elements.

How to Multiply Matrices:

Given two matrices **A** and **B**, the resulting matrix **C** = **A** × **B** will have dimensions based on the **rows of A** and the **columns of B**.

- If matrix **A** has dimensions (m x n) (m rows and n columns) and matrix **B** has dimensions (n x p) (n rows and p columns), the resulting matrix **C** will have dimensions (m x p).

The formula for the element c_{ij} in the resulting matrix **C** is:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

This means you multiply each element of the **i-th** row of **A** by the corresponding element of the **j-th** column of **B** and sum the results.

Example of Matrix Multiplication:

Let's multiply the following matrices:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

To find the product $A \times B$, we calculate each element of the resulting matrix C .

1. **First row, first column (c_{11}):**

$$c_{11} = (1 \times 5) + (2 \times 7) = 5 + 14 = 19$$

2. **First row, second column (c_{12}):**

$$c_{12} = (1 \times 6) + (2 \times 8) = 6 + 16 = 22$$

3. **Second row, first column (c_{21}):**

$$c_{21} = (3 \times 5) + (4 \times 7) = 15 + 28 = 43$$

4. **Second row, second column (c_{22}):**

$$c_{22} = (3 \times 6) + (4 \times 8) = 18 + 32 = 50$$

So the resulting matrix C is:

$$C = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

Python Example

```
In [ ]: matrix_mult = np.dot(matrix_a, matrix_b)

print("Matrix Multiplication:\n", matrix_mult)
```

```
Matrix Multiplication:
[[19 22]
 [43 50]]
```

Explanation:

- `np.dot(matrix_a, matrix_b)` : Multiplies matrices using the dot product.

Matrix Inversion and Determinants

Determinants

A **determinant** is a **scalar** (single number) that can be calculated from a **square matrix**. The determinant of a matrix provides important information about the matrix, such as:

- Whether the matrix is **invertible** (a matrix is invertible if its determinant is non-zero).

Formula to Calculate Determinants:

For a 2x2 matrix, the determinant is calculated as:

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = (a \cdot d) - (b \cdot c)$$

For a 3x3 matrix, the determinant is calculated using a more complex formula:

$$\det(A) = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \cdot \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \cdot \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \cdot \begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

Example of Determinant (2x2 Matrix):

Given the matrix:

$$A = \begin{pmatrix} 2 & 3 \\ 4 & 5 \end{pmatrix}$$

The determinant of matrix **A** is:

$$\det(A) = (2 \cdot 5) - (3 \cdot 4) = 10 - 12 = -2$$

Example of Determinant (3x3 Matrix):

Given the matrix:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 1 & 0 & 6 \end{pmatrix}$$

The determinant of matrix **A** is:

$$\begin{aligned} \det(A) &= 1 \cdot \begin{vmatrix} 4 & 5 \\ 0 & 6 \end{vmatrix} - 2 \cdot \begin{vmatrix} 0 & 5 \\ 1 & 6 \end{vmatrix} + 3 \cdot \begin{vmatrix} 0 & 4 \\ 1 & 0 \end{vmatrix} \\ &= 1 \cdot (4 \cdot 6 - 5 \cdot 0) - 2 \cdot (0 \cdot 6 - 5 \cdot 1) + 3 \cdot (0 \cdot 0 - 4 \cdot 1) \\ &= 1 \cdot (24) - 2 \cdot (-5) + 3 \cdot (-4) \\ &= 24 + 10 - 12 = 22 \end{aligned}$$

So, the determinant of **A** is 22.

Importance of Determinants

1. Invertibility:

- If the determinant of a matrix is **0**, the matrix is **singular** and **not invertible**.
- If the determinant is **non-zero**, the matrix is **invertible**.

2. Geometric Interpretation:

- The determinant represents the **scaling factor** by which the matrix stretches or shrinks space.
- A determinant of **1** means the matrix does not change the **area** or **volume**.
- Larger or smaller values indicate how much the matrix **scales** the space.

Python Example

```
In [ ]: determinant = np.linalg.det(matrix_a)

print("Determinant:", determinant)
```

Determinant: -2.0000000000000004

Explanation:

- `np.linalg.det(matrix_a)` : Calculates the determinant of `matrix_a`.

Why Determinants Matter in Data Science

1. Data Quality:

- Determinants help detect issues like **multicollinearity** (too much similarity between features), which can lead to poor model performance.

2. Model Stability:

- A non-zero determinant ensures that your model is stable and avoids unreliable results caused by singular or poorly conditioned matrices.

3. Simplifying Data:

- In **dimensionality reduction** (e.g., PCA), determinants indicate how much **information is preserved** when reducing the data's complexity.

Matrix Inversion

What is a Matrix Inverse?

- The **inverse of a matrix** is a special matrix that, when multiplied with the original matrix, gives the **identity matrix**.
- Identity Matrix is a square matrix in which all the elements of principal diagonals are one, and all other elements are zeros.

Formula for Matrix Inversion

For a **2x2 matrix** A , the inverse is calculated as:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

The inverse A^{-1} is given by:

$$A^{-1} = \frac{1}{\det(A)} \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Where $\det(A) = ad - bc$ is the **determinant** of the matrix.

- If $\det(A) = 0$, the matrix is **not invertible**.

Shortcut to Remember the Formula for Matrix Inversion

1. **Swap the diagonal elements:** Exchange the positions of (a) and (d).
2. **Change the signs of the off-diagonal elements:** Negate (b) and (c).
3. **Divide by the determinant det(a)**

Example of Matrix Inversion:

For the matrix:

$$A = \begin{pmatrix} 2 & 3 \\ 1 & 4 \end{pmatrix}$$

1. Calculate the determinant:

$$\det(A) = (2 \cdot 4) - (3 \cdot 1) = 8 - 3 = 5$$

2. Apply the formula:

$$A^{-1} = \frac{1}{5} \cdot \begin{pmatrix} 4 & -3 \\ -1 & 2 \end{pmatrix}$$

3. Result:

$$A^{-1} = \begin{pmatrix} 0.8 & -0.6 \\ -0.2 & 0.4 \end{pmatrix}$$

Importance of Matrix Inversion in Data Science

1. Solving Systems of Equations

- In regression (such as **linear regression**), the inverse of the design matrix is used to find the best-fit coefficients.

2. Dimensionality Reduction

- Techniques like **Principal Component Analysis (PCA)** often use matrix inverses when calculating **eigenvalues** and **eigenvectors**.

3. Optimization

- Many machine learning algorithms, such as **Gradient Descent** or **Newton's Method**, rely on matrix inversion to solve optimization problems efficiently.

4. Transformations

- In **computer graphics** and **data transformations**, matrix inversion is used to undo transformations like **rotations** or **scaling**.

Python Example

```
In [ ]: matrix_inv = np.linalg.inv(matrix_a)

print("Inverse Matrix:\n", matrix_inv)
```

Inverse Matrix:

```
[[ -2.   1. ]
 [ 1.5 -0.5]]
```

Explanation:

- `np.linalg.inv(matrix_a)` : Computes the inverse of a matrix.

Transpose Of Matrix

What is the Transpose of a Matrix?

The transpose of a matrix is a new matrix obtained by flipping the matrix over its diagonal. In simple terms:

- The rows of the original matrix become the columns of the transposed matrix.
- The columns of the original matrix become the rows of the transposed matrix.

If a matrix A has elements a_{ij} (where i is the row number and j is the column number), the transpose of A , denoted as A^T , has its elements rearranged such that:

$$(A^T)_{ij} = A_{ji}$$

Example

Let A be a 2×3 matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

The transpose of A , denoted as A^T , is obtained by switching rows and columns:

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Here:

- The first row of A $[1, 2, 3]$ becomes the first column of A^T .
- The second row of A $[4, 5, 6]$ becomes the second column of A^T .

Properties of Transpose

1. Transpose of a Transpose:

- If you transpose a matrix twice, you get back the original matrix:
$$(A^T)^T = A$$

2. Transpose of a Sum:

- The transpose of the sum of two matrices is the sum of their transposes:
$$(A + B)^T = A^T + B^T$$

3. Transpose of a Product:

- The transpose of the product of two matrices reverses the order of multiplication:
$$(AB)^T = B^T A^T$$

4. Transpose of a Scalar Multiplication:

- Multiplying a matrix by a scalar and then transposing it is the same as transposing the matrix first and then multiplying by the scalar:
$$(cA)^T = c(A^T)$$

5. Symmetric Matrix:

- A matrix is symmetric if it is equal to its transpose:
$$A = A^T$$

How to Compute Transpose in Python

```
In [ ]: import numpy as np

# Define a matrix
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# Compute transpose
A_transpose = A.T

print("Original Matrix:")
print(A)

print("\nTranspose of the Matrix:")
print(A_transpose)
```

Original Matrix:

```
[[1 2 3]
 [4 5 6]]
```

Transpose of the Matrix:

```
[[1 4]
 [2 5]
 [3 6]]
```

System Of Linear Equations

What is a System of Linear Equations?

- A system of linear equations is a group of two or more equations that involve the same set of variables.
- These equations are "linear," which means they describe straight lines if graphed.
- The goal is to find the values of these variables that satisfy all the equations at the same time.

Key Features of a Linear Equation

A linear equation has the following characteristics:

1. **Variables are simple:**

- Variables appear in their simplest form, not squared, cubed, or inside functions like sine, cosine, or logarithms.

2. **Straight line:**

- When plotted, the equation forms a straight line in 2D or a flat plane in 3D.

3. Examples:

- $2x + 3y = 5$ (linear, two variables).
- $x - 4y + 2z = 7$ (linear, three variables).
- $x^2 + y = 3$ (not linear, because x^2 is squared).

Real-Life Example of a System of Linear Equations

Imagine you're buying apples and bananas:

- **First equation:** The total cost of 2 apples and 3 bananas is ₹50:

$$2x + 3y = 50$$

- **Second equation:** The total cost of 1 apple and 2 bananas is ₹30:

$$x + 2y = 30$$

Here, x is the price of an apple, and y is the price of a banana.

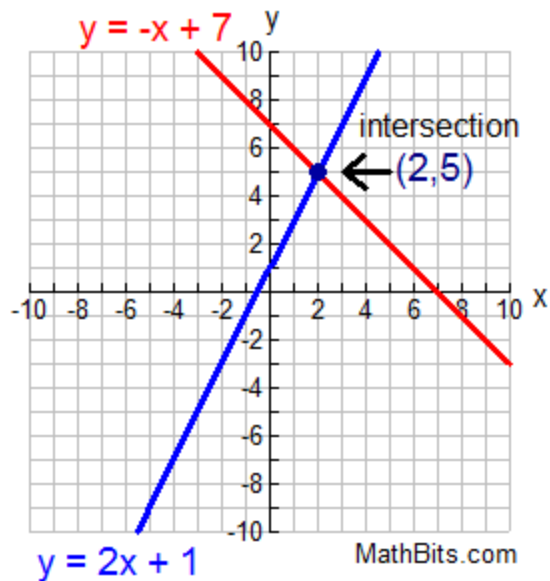
The solution to this system is the values of x and y that make both equations true.

For example, if $x = 10$ and $y = 10$, both equations are satisfied.

Methods to Solve Systems of Linear Equations: Step-by-Step

1. Graphical Method

- **How It Works:**
 - Plot each equation as a line on a graph.
 - The intersection point(s), if any, represent the solution(s).



- **Advantages:**

- Intuitive and visual.

- **Disadvantages:**

- Can be inaccurate for complex systems or when precise solutions are needed.

2. Substitution Method

- **How It Works:**

- Solve one equation for one variable (e.g., $y = \dots$).
- Substitute this expression into the other equation(s).
- Solve for the remaining variable(s).

- **Example:**

Using the system:

$$1) 2x + y = 5$$

$$2) x - y = 1$$

- Solve $x - y = 1$ for y : $y = x - 1$.
- Substitute $y = x - 1$ into $2x + y = 5$:
 $2x + (x - 1) = 5$
 $3x = 6$, so $x = 2$.
- Substitute $x = 2$ back into $y = x - 1$:
 $y = 2 - 1$, so $y = 1$.
- **Solution:** $x = 2, y = 1$.

3. Elimination Method

- How It Works:
 - Add or subtract equations to eliminate one variable.
 - Solve the resulting equation for the remaining variable.

- **Example:**

Using the system:

$$1) 2x + y = 5$$

$$2) x - y = 1$$

- Add the two equations:
 $2x + y + x - y = 5 + 1$
 $3x = 6$, so $x = 2$.
- Substitute $x = 2$ into $x - y = 1$:
 $2 - y = 1$, so $y = 1$.
- **Solution:** $x = 2, y = 1$.

Matrix Methods for Solving Linear Systems

In linear algebra, matrix methods are a powerful way to solve systems of linear equations. Let's break it down step by step, using simple words and an easy-to-follow explanation.

1. Understanding the Problem

You have a system of linear equations, something like:

$$\begin{aligned} 2x + 3y &= 5 \\ 4x + y &= 6 \end{aligned}$$

This system involves two equations and two unknowns (x and y). Instead of solving each equation separately, we can represent this system using matrices.

2. Writing the System as a Matrix Equation

To represent the system as a matrix equation, we first rewrite the system in a standard form:

Coefficients of x	2	4
Coefficients of y	3	1
Constants (right-hand side)	5	6

Now, let's break it into three parts:

- **A** : The matrix of coefficients (the numbers in front of the variables).
- **x** : The column matrix representing the variables (the unknowns you want to solve for).
- **b** : The column matrix of constants (the numbers on the right-hand side of the equations).

We can write this as:

$$A \cdot x = b$$

Where:

- $A = \begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix}$
- $x = \begin{bmatrix} x \\ y \end{bmatrix}$
- $b = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$

This matrix equation is a compact way to represent the system of linear equations.

Eigen Values and Eigen Vectors

What Are Eigenvalues and Eigenvectors?

1. Eigenvectors:

- Vectors are arrows that point in a certain direction and have a certain length.
- Eigenvectors are special vectors that, when we apply a transformation (like rotating, stretching, or shrinking) to them, **do not change their direction**. They still point in the same direction as before. The only thing that happens is they might get longer, shorter, or flip.

Example:

- Imagine you have a vector pointing to the right (along the x-axis). When you apply a transformation (like stretching the space), this vector might become longer, but it will still point to the right (along the x-axis).
- If this vector changes its direction, it's **not an eigenvector**.
- So, eigenvectors stay on their "path" or "line" even after the transformation.

2. Eigenvalues:

- An **eigenvalue** is simply a number that tells you **how much an eigenvector stretches** or shrinks when the transformation is applied.
- Think of it like this:
 - If the transformation **stretches** the eigenvector, the eigenvalue will be a number **greater than 1**.

- If the transformation **shrinks** the eigenvector, the eigenvalue will be a number **less than 1**.
- If the transformation **flips** the eigenvector (like flipping it upside down), the eigenvalue will be a **negative number**.

Example:

- If you have an eigenvector pointing to the right (along the x-axis), and the transformation **doubles** its length, the eigenvalue would be **2** (because the vector is stretched by a factor of 2).
- If the transformation **shrinks** the vector by half, the eigenvalue would be **0.5**.
- If the vector **flips upside down** (still on the same line, but in the opposite direction), the eigenvalue would be **-1** (because it's flipped).

Key Idea:

- When you multiply a matrix by one of its eigenvectors, the result is the same vector scaled by its eigenvalue.

Mathematically:

$$A \cdot \mathbf{v} = \lambda \cdot \mathbf{v}$$

- A : A matrix (represents a transformation)
- $\mathbf{v}$: An eigenvector
- λ : The eigenvalue

This means:

- $A \cdot \mathbf{v}$ (the result of transforming the vector $\mathbf{v}$) is the same as scaling $\mathbf{v}$ by the factor λ (eigenvalue).

Simple Analogy:

Think of a rubber sheet (representing space) and a matrix as a set of hands stretching or squishing the sheet.

- Some points on the sheet stay aligned along the same direction, even if they stretch or shrink. These are the **eigenvectors**.
- The amount by which they stretch or shrink is the **eigenvalue**.

Simple Example

Imaging a matrix

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$$

This matrix can be thought of as a transformation machine. Its job is to take a vector, do some stretching or squishing, and give you a new vector.

Now, we are applying this matrix to two special vectors:

1. **Vector 1:** $v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ (points directly along the x-axis).
2. **Vector 2:** $v_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ (points directly along the y-axis).

These vectors are special because they only point along one axis at a time, making it easy to see how the transformation affects them.

Step 1: Apply the Matrix to Vector 1

We multiply the matrix A with v_1 :

$$A \cdot v_1 = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + 3 \cdot 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

- **What happened?**
The vector v_1 was stretched along the x-axis by a factor of 2.
- **Eigenvalue:** The eigenvalue for this vector is 2, because it scales by 2.

Step 2: Apply the Matrix to Vector 2

We multiply the matrix A with v_2 :

$$A \cdot v_2 = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \cdot 0 + 0 \cdot 1 \\ 0 \cdot 0 + 3 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 3 \end{pmatrix}$$

- **What happened?**
The vector v_2 was stretched along the y-axis by a factor of 3.
- **Eigenvalue:** The eigenvalue for this vector is 3, because it scales by 3.

What Does This Mean?

1. Eigenvectors:

- $v_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $v_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ are **eigenvectors** because their direction doesn't change after the transformation.
- They still point along the x-axis and y-axis, respectively.

2. Eigenvalues:

- For v_1 , the eigenvalue is 2, meaning it stretches by a factor of 2 along the x-axis.
- For v_2 , the eigenvalue is 3, meaning it stretches by a factor of 3 along the y-axis.

Why Are Eigenvalues and Eigenvectors Important?

Eigenvalues and eigenvectors might seem abstract, but they are incredibly useful tools in many areas of science, mathematics, and engineering. Let's break it down with simple explanations and practical applications:

1. Data Compression (e.g., PCA)

When working with large datasets, like images, videos, or massive spreadsheets, it's often important to reduce the data size while keeping the most useful information.

What happens in PCA?

- **PCA (Principal Component Analysis)** uses eigenvalues and eigenvectors to find the "most important directions" in the data.
- These directions, represented by **eigenvectors**, show where the data varies the most.
- **Eigenvalues** tell us how much variation happens in each direction.

Example:

Imagine you're analyzing hundreds of photos. Most of the variation might come from brightness or one dominant pattern (e.g., landscape vs. portrait). PCA will identify this variation and allow you to compress the data by keeping only the key patterns.

Why does it matter?

- This helps in reducing the size of datasets.
- Speeds up machine learning algorithms.
- Removes redundant information.

2. Machine Learning and Artificial Intelligence

Eigenvalues and eigenvectors are at the heart of many machine learning algorithms. They help simplify the data and make models more efficient.

How are they used?

- **Data Simplification:** In machine learning, high-dimensional data (e.g., thousands of features in a dataset) is reduced to fewer dimensions using eigenvectors. This reduces complexity without losing important information.
- **Stability and Optimization:** Algorithms like linear regression, SVMs, and neural networks rely on matrix operations where eigenvalues are used to check the stability of solutions and optimize models.

Why does it matter?

- Enables faster, more accurate, and more efficient machine learning models.
- Critical for tasks like speech recognition, recommendation systems, and autonomous vehicles.



THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)